

Perfectly secure cryptosystems exist:

Ex (one-time pads):

let $P = C = X = \mathbb{Z}_n^d$ for any values of $d, n \geq 1$.

Define $e_k: P \rightarrow C$, $d_k: C \rightarrow P$ via $e_k(x) = x + k$ and $d_k(y) = y - k$ with operations in $\mathbb{Z}_n^d$, i.e. modulo n in each component.

Assuming that keys are chosen uniformly at random, the cryptosystem is perfectly secure by Shannon's Theorem.

Namely, condition (a) is satisfied by construction and for $x, y \in \mathbb{Z}_n^d$, the unique key $k \in \mathbb{Z}_n^d$ with $e_k(x) = x + k = y$

i) $k = y - x$

For group theorists: If G is any finite group, we obtain a perfectly secure cryptosystem with $P = C = X = G$, $e_k(x) = kx$ and $d_k(y) = k^{-1}y$. The example above is the case $G = (\mathbb{Z}_n^d, +)$.

39

Special case: A Vigenère cipher where the key used
is a random string of the same length
as the message.

This illustrates a major issue of perfectly secure cryptosystems:
we need to securely exchange a key that is as large
as the actual message.

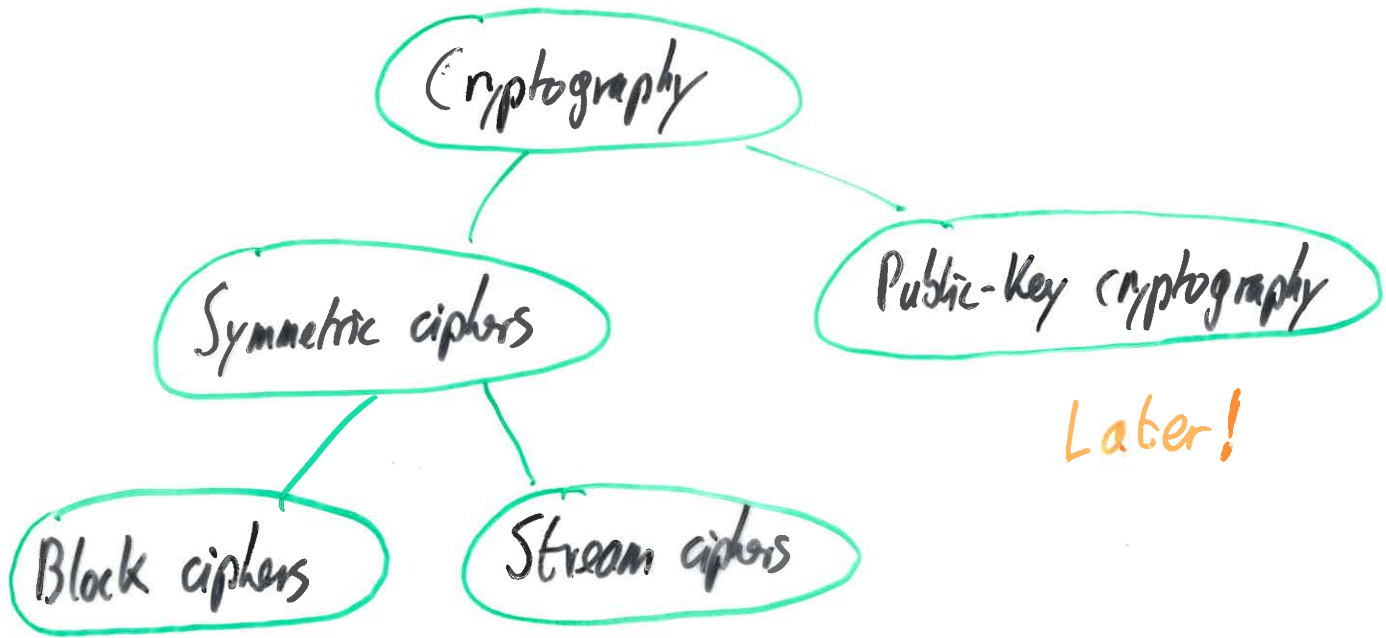
Another issue: Perfect security requires keys to never be
reused, except by chance!

Suppose, for example, that we encrypt several plaintexts
 $x_1, x_2, \dots \in \mathbb{Z}_n^d$ using the same key $K \in \mathbb{Z}_n^d$ to
produce $y_i = x_i + K$. We may then recover

$$x_i - x_j = y_i - y_j$$

which may reveal something about the x_i .

A map of cryptography



Block ciphers operate on blocks of plaintext and ciphertext symbols to produce blocks of ciphertext and plaintext symbols, respectively.

Ex: Affine ciphers of arbitrary dimension d :

$$\mathcal{P} = \mathcal{C} = \mathbb{Z}_n^d = \text{set of blocks of length } d \text{ over } \mathbb{Z}_n$$

Real-world examples:

- DES (Data Encryption Standard) 1970s:
block size of 64 bits, key size of 56 bits ... insecure!
- AES (Advanced Encryption Standard) ~1998:
block size of 128 bits, possible key sizes of 128, 192, or 256 bits.
Generally regarded as secure. Hardware support: → AES instruction set

Stream ciphers operate on "streams" of symbols over an alphabet. Each (plaintext) symbol in the input stream is encrypted in a way that depends on the previous "history" of the stream.

Usually: • Plaintexts are streams $a_1, a_2, \dots$ over $\mathbb{Z}_n$.

(In practice: $\mathbb{Z}_2$)

- Keys are "seeds" used to generate "pseudo-random"
 $\approx$ "looks random"

Key streams $k_1, k_2, \dots$ over $\mathbb{Z}_n$ by means of a deterministic procedure.

- Encryption: $a_1, a_2, \dots \rightsquigarrow a_1 + k_1, a_2 + k_2, \dots$
- Decryption: subtraction

Notes:

- Stream ciphers use the same idea as the one-time pad except that a truly random key stream is replaced by a deterministic process that merely seeks to appear random.
 Is this possible?

- Synchronous stream ciphers: the key stream is generated⁴² independently of the plaintext
- Asynchronous stream ciphers: K_n might depend on $a_1 \rightarrow a_n$, in addition to $K_1 \rightarrow K_{n-1}$.
- Stream ciphers tend to be small, fast, and easily implementable, both in hardware and in software. They are commonly used when computational resources are limited.

Ex: • "A5/1" is used in the GSM (Global System for Mobile communications) standard ("2G")

... very insecure nowadays! → Snowden leaks 2013

- Block ciphers are commonly used to secure internet communication; they are often combined with elements of Public-Key cryptography.
- The security of a stream cipher rests entirely on our ability to generate "cryptographically good" pseudo-random key streams.

In the following, all streams will be binary, i.e. over $\mathbb{Z}_2$.⁴³

Thm: no real restriction since we can encode the numbers $0, \dots, n-1$ using bit strings of length $\leq \lceil \log_2 n \rceil$ by taking binary expansions.

Ex: $n=6$ so $\lceil \log_2 6 \rceil = 3$ bits per symbol from $\mathbb{Z}_6$

0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1

Note: Addition $+_2$ and multiplication $\cdot_2$ in $\mathbb{Z}_2$ are exactly the bitwise/logical operations XOR and AND, resp.

These correspond to standard $\rightarrow$ Logic gates

$+_2$	0	1
0	0	1
1	1	0

$\cdot_2$	0	1
0	0	0
1	0	1

Fun fact: All Boolean functions (i.e. functions $\mathbb{Z}_2^d \rightarrow \mathbb{Z}_2$) can be expressed in terms of polynomials over $\mathbb{Z}_2$, e.g. $\neg x = x+1$,
 $x \text{ OR } y = x + y + xy, \dots$

Defn A feedback shift register of length L is a particular type of machine (abstract or real).

The state of the register is a bit string $s_{L-1}, \dots, s_1, s_0$ of length L . Given an initial state, the operation of the register is governed by a clock. During each clock cycle,

① the current value of s_0 is appended to the output stream and

② the current state $s_{L-1}, \dots, s_0$ is then replaced by

$$f(s_{L-1}, \dots, s_0), s_{L-1}, \dots, s_1$$

where $f: \mathbb{Z}_2^L \rightarrow \mathbb{Z}_2$ is the feedback function of the register.

Picture:

